Smart Charge Reverse Engineering

Huawei MateBook D 15 (BohrD-WDH9D) – ACPI / Embedded Controller analysis and Linux implementation

Extended version (publication-style) • Date: 2026-03-07

Author: Guido Sorarù (sorguido@gmail.com)

General Index

Abstract....1
1. System and context....1
2. Goal and problem statement....2
3. Reverse engineering methodology....2
3.1 ACPI table extraction....2
3.2 Method discovery and EC correlation....2
3.3 Experimental validation....2
4. Relevant ACPI methods....2
5. EC registers and observed semantics....2
5.1 Diagrams....3
6. Key experimental results....4
6.1 Working sequence (40/70)....4
6.2 EC registers and sysfs observation....4
7. Persistence: reboot vs shutdown....4
8. Tooling: Smart_Charge.py....5
Appendix A — Tested ACPI commands (complete extract)....5
Appendix B — Research timeline....6
9. Conclusions and future work....7

Abstract

This paper documents a complete reverse engineering effort of the battery charge limiting mechanism (“Smart Charge” / Battery Protection) implemented in Huawei MateBook firmware. The practical goal is to reproduce on Linux the Windows Huawei PC Manager behavior, configuring charge thresholds (e.g., stop at 70%) and characterizing platform constraints.

The work includes ACPI table extraction and decompilation (DSDT/SSDT), identification of relevant AML methods, correlation with Embedded Controller (EC) registers, experimental validation via acpi_call, and persistence analysis (reboot vs shutdown).

1. System and context

Hardware: Huawei MateBook D 15 (BohrD-WDH9D), year 2021.
Operating system: Linux Mint. Kernel: 6.x series (tested on 6.17).
ACPI battery device: BAT1.
Embedded Controller ACPI path:
_SB.PC00.LPCB.EC0.

2. Goal and problem statement

Functional goal: control the battery upper charge limit (e.g., stop at 70%) similarly to Huawei PC Manager on Windows. Constraints: Windows removed (no direct WMI sniffing), must operate via ACPI/EC on Linux.

Initial issue: the Linux huawei_wmi driver exposes a sysfs node for charge thresholds, but writes fail with I/O error, effectively allowing read-only access.

3. Reverse engineering methodology

3.1 ACPI table extraction

ACPI tables were extracted using acpidump and split with acpixtract. AML was decompiled to DSL via iasl. The decompilation produced AE_NOT_FOUND / External warnings due to cross-table references (common with extracted sets), but the resulting DSL is usable for static analysis.

3.2 Method discovery and EC correlation

Candidate methods were identified by name (SBTT/GBTT/SBCM/GBCM) and confirmed by locating EC register writes in DSL. The correlation was validated experimentally by reading EC registers before and after method execution.

3.3 Experimental validation

Runtime execution was performed using the acpi_call-dkms module.
Testing consisted of:
- invoking methods with specific buffers,
- reading /proc/acpi/call returns,
- observing EC registers (CHAM/BCCL/BCCH…),
- verifying effect through /sys/class/power_supply/BAT1/* (status/capacity/current_now).

4. Relevant ACPI methods

Main methods identified in DSDT and/or routed through the WMI table:
- \SBTT: set battery thresholds (writes BCCL/BCCH) - \GBTT: get thresholds
- \SBCM: set charge mode (writes CHAM and may update thresholds)
- \GBCM: get current mode/parameters
- \SBAC / \GBAC: AC-related controls (observed but not decisive)
- \SODP / \SPLV /\SSAP / \SSUP / \SBPP: tested as possible unlock/override candidates (no observable effect on charge limits)

5. EC registers and observed semantics

Static and experimental evidence highlights the following key EC registers:

CHAM (8-bit): charge mode selector (“charge mode”). BCCL (8-bit): lower threshold (often 0x28=40). BCCH (8-bit): upper threshold (typical 0x46=70). BADS (bit): AC disable/state flag.

SPMD (bit): power management mode involved in specific DSDT code paths.

Important note: in real tests the lower threshold does not gate charging start: if SOC is 50% and AC is plugged, charging starts immediately and proceeds until the upper limit.

5.1 Diagrams

Figure 1 — Control chain OS → ACPI → EC → firmware → charger → battery.

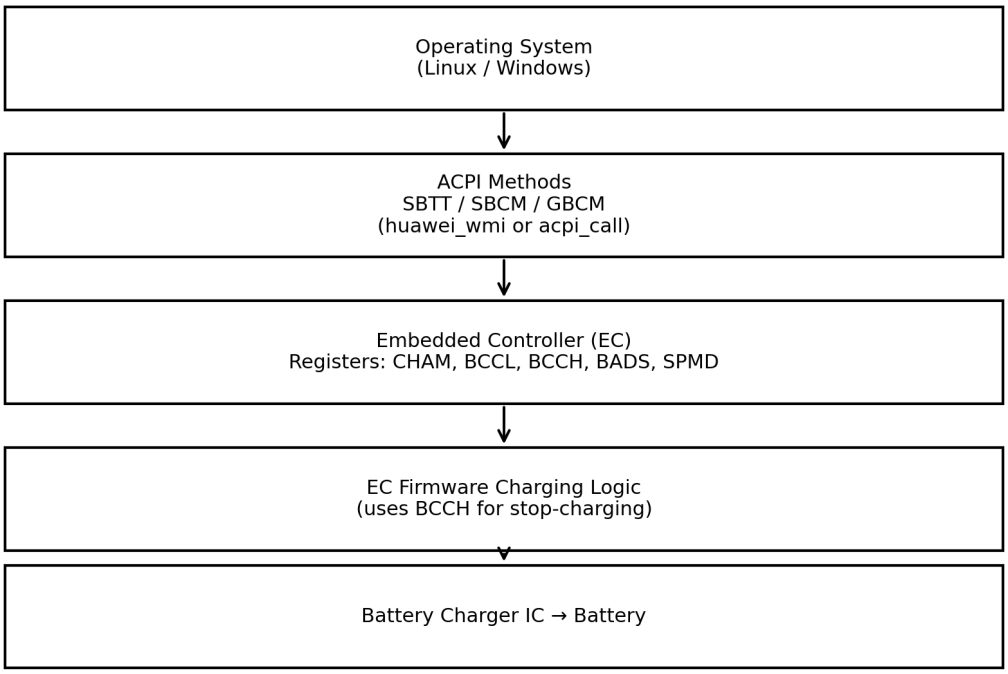


Figure 2 — Simplified charging state machine.

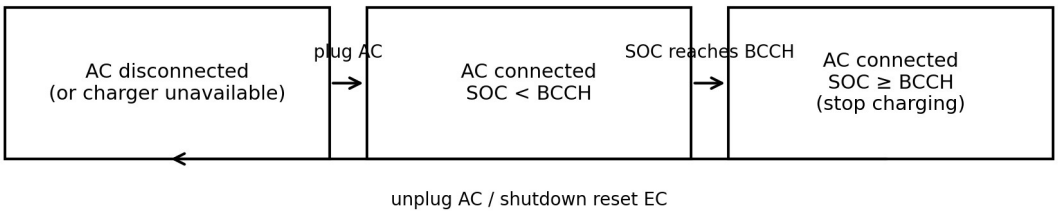


Figure 3 — Relevant EC register map.

Register	Meaning	Observed / Typical values
CHAM	Charge mode / policy selector	0x04 (Smart Charge active); 0x00 (limits cleared)
BCCL	Lower threshold parameter	0x28 (40%) — stored, but does not gate charging
BCCH	Upper threshold parameter	0x46 (70%) — used to stop charging
BADS	AC disable / battery AC state flag	0x00 in observed snapshots
SPMD	Power management mode flag	0x00 in observed snapshots

6. Key experimental results

6.1 Working sequence (40/70)

A stable sequence was identified to configure “Smart Charge” with stop at 70%.

The sequence uses:

1) \SBTT to write thresholds
2) \SBCM to set CHAM and confirm parameters
3) \GBCM to read back resulting state

```
sudo modprobe acpi_call
echo '\SBTT (Buffer(){0x00,0x00,0x28,0x46})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SBCM (Buffer(){0x00,0x00,0x01,0x00,0x28,0x46})' | sudo tee
/proc/acpi/call
sudo cat /proc/acpi/call
echo '\GBCM (Buffer(){0x00})' | sudo tee /proc/acpi/call >/dev/null
sudo cat /proc/acpi/call
```

6.2 EC registers and sysfs observation

EC registers match the active mode: CHAM=0x04, BCCL=0x28, BCCH=0x46.
At sysfs level, once 70% is reached:
BAT1/status → Not charging BAT1/current_now → 0

7. Persistence: reboot vs shutdown

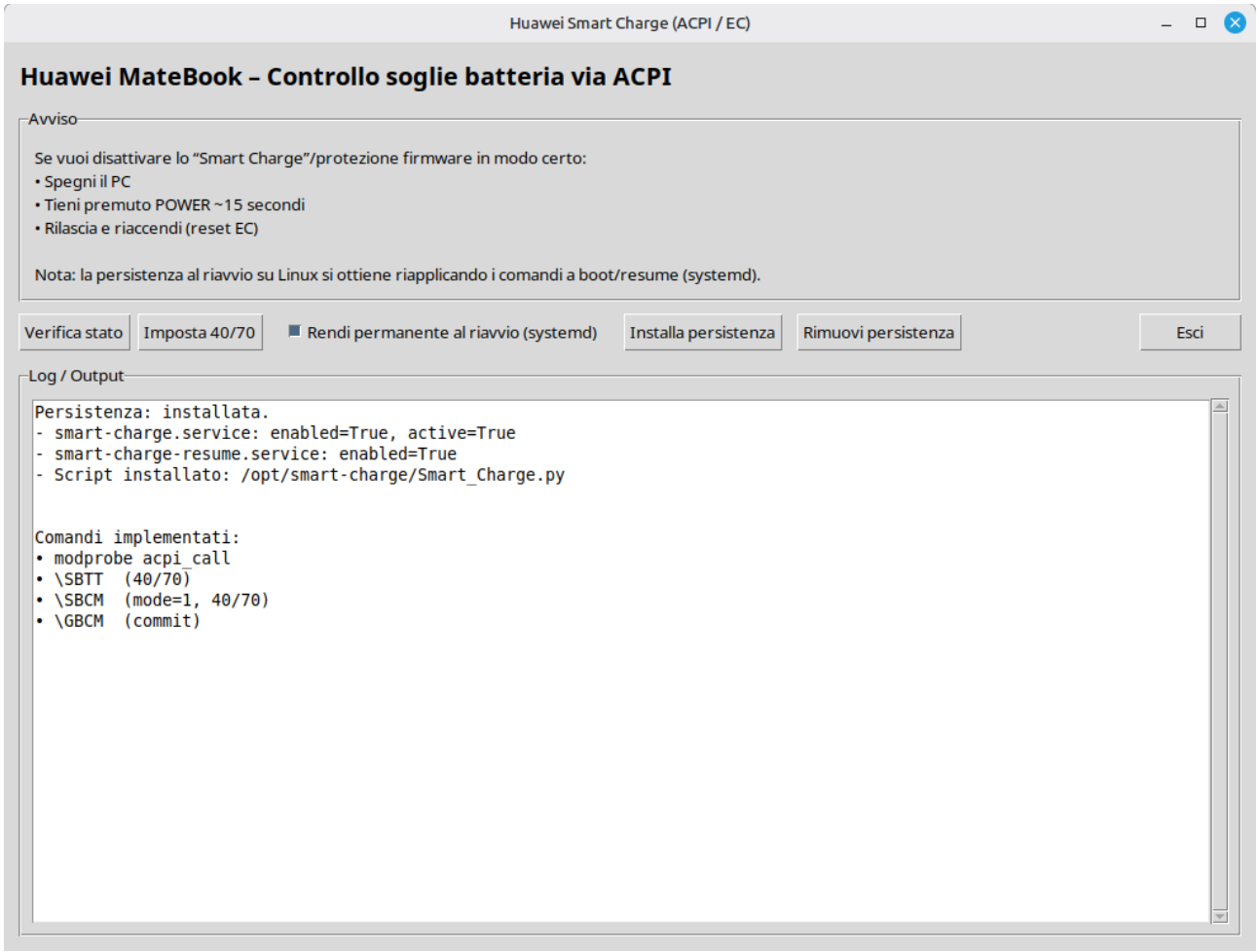
Key finding: “Smart Charge” settings persist across reboot, but not across a full shutdown. Technical interpretation: the configuration appears to live in volatile EC state or a runtime context not stored persistently.

Implication: to keep the limit “always on”, applying it at every boot (systemd/autostart) is sufficient.

8. Tooling: Smart_Charge.py

A Python GUI utility (Smart_Charge.py) was developed to automate:
- loading the acpi_call module
- executing the working sequence
- reading back ACPI return values and battery parameters
- user notice: to fully disable

Smart Charge, powering off and holding the power button ~15s (EC reset) may be required.
Note: the GUI requires a graphical session (DISPLAY set).
In headless environments Tk will raise TclError “no display name”.



Appendix A — Tested ACPI commands (complete extract)

Below is a consolidated list of ACPI commands actually used during testing and referenced across the research.
This list is useful to reproduce the experiment and for further firmware/EC investigation.

```
# Load module
sudo modprobe acpi_call

# Working configuration (40/70)
echo '\SBTT (Buffer(){0x00,0x00,0x28,0x46})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SBCM (Buffer(){0x00,0x00,0x01,0x00,0x28,0x46})' | sudo tee
/proc/acpi/call
sudo cat /proc/acpi/call
echo '\GBCM (Buffer(){0x00})' | sudo tee /proc/acpi/call >/dev/null
sudo cat /proc/acpi/call
echo '\GBTT (Buffer(){0x00,0x00,0x00,0x00})' | sudo tee /proc/acpi/call
>/dev/null
sudo cat /proc/acpi/call

# Attempts to set 0/100 (did not persist as expected in Smart Charge
mode)
echo '\SBTT (Buffer(){0x00,0x00,0x00,0x64})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SBCM (Buffer(){0x00,0x00,0x01,0x00,0x00,0x64})' | sudo tee
/proc/acpi/call
sudo cat /proc/acpi/call

# Read EC registers directly
echo '\_SB.PC00.LPCB.EC0.CHAM' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.BCCL' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.BCCH' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.BADS' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
echo '\_SB.PC00.LPCB.EC0.SPMD' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call

# Other candidates tested (no effect observed)
echo '\SODP (Buffer(){0x00,0x00,0x00,0x00})' | sudo tee /proc/acpi/call
>/dev/null
echo '\SPLV (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SPLV (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SPLV (0x02)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSAP (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSAP (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSAP (0x02)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSUP (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSUP (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SSUP (0x02)' | sudo tee /proc/acpi/call >/dev/null
echo '\SBPP (0x00)' | sudo tee /proc/acpi/call >/dev/null
echo '\SBPP (0x01)' | sudo tee /proc/acpi/call >/dev/null
echo '\SBPP (0x02)' | sudo tee /proc/acpi/call >/dev/null

# AC behavior (read)
echo '\GBAC (Buffer(){0x00})' | sudo tee /proc/acpi/call >/dev/null;
sudo cat /proc/acpi/call
```

Appendix B — Research timeline

- Phase 1: investigate huawei_wmi driver; threshold writes fail (I/O error)
- Phase 2: extract ACPI (acpidump/acpixtract) and decompile (iasl)
- Phase 3: identify SBTT/GBTT/SBCM/GBCM methods and EC registers
- Phase 4: runtime testing via acpi_call; validate 40/70 thresholds

- Phase 5: analyze 70% lock and “firmware mode” hypothesis
- Phase 6: discover persistence behavior (reboot keeps, shutdown resets)
- Phase 7: develop Smart_Charge.py and documentation

9. Conclusions and future work

Conclusion:
Linux can reproduce the upper charge limit (stop at 70%) via ACPI methods that write EC registers. The lower threshold appears unused for charging start gating, at least on this platform and observed behavior.
The mechanism is not persistent across shutdown, so applying it at boot is the correct approach for consistent behavior.

Recommended future work:
- stable integration into the huawei_wmi driver (enable threshold writes)
- formalization of the WMI protocol (HWMI/WMAA) if additional controls are needed
- evaluation of EC events and BAT1 notify paths for state updates.